

Elastic Horizon: Discovering the Effective Interaction Frontier in Agentic Reinforcement Learning

Gangyi Zhang^{1*}, Junjie Meng^{2*}, Letian Zhang³, Wei Wu², Yang Zheng³,
Dong Wang^{1†}, Yang Liu¹, Guanjun Jiang¹, Chongming Gao^{2†}

¹Qwen Business Unit of Alibaba, ²University of Science and Technology of China,
³Independent Researcher

gangyi.cn@gmail.com, mengjre@gmail.com
wd443495@alibaba-inc.com, chongming.gao@gmail.com

Abstract

Scaling the interaction horizon—the maximum number of environment interactions per episode—improves LLM agents on long-horizon tasks, and curriculum-based methods that progressively expand the horizon outperform fixed-horizon alternatives. However, existing schedules are *open-loop*: they monotonically increase the horizon until a manually specified maximum, with no mechanism to detect when further expansion stops helping. We propose the *effective interaction frontier hypothesis*: a dynamic boundary beyond which additional interactions yield diminishing returns while cost grows linearly. We then introduce **Elastic Horizon**, a closed-loop controller that tracks this boundary via the 90th percentile of successful trajectory lengths. On AppWorld and BFCL, fixed-horizon sweeps reveal clear saturation plateaus; Elastic Horizon stabilizes the horizon inside the saturation band from both under- and over-capacity initializations, attains the best success rates across 7B and 14B backbones, and saves up to 25% of per-step trajectory tokens. Our work shifts the paradigm from *how to scale* interaction horizons to *when to stop scaling*.

1 Introduction

In partially observable environments, LLM agents must gather information through multi-turn interactions to reduce state uncertainty and complete tasks (Yao et al., 2023; Shinn et al., 2023). Recent work demonstrates that *scaling the interaction horizon*—the maximum number of environment interactions per episode—substantially improves performance on long-horizon agentic tasks. Xi et al. (2025) introduce ScalingInter-RL, a curriculum that linearly increases the horizon during training, matching or surpassing commercial models

on 27 tasks across diverse environments. Shen et al. (2025) show that a multiplicative schedule outperforms both fixed-horizon training and additive schedules on web navigation benchmarks.

Despite their success, these approaches share a fundamental characteristic: they are *open-loop monotonically increasing* schedules. The horizon grows according to a predetermined function of training steps, continuing until reaching a manually specified maximum $K_{\max}$, with no mechanism to assess whether further expansion remains beneficial. This design implicitly assumes that performance improves monotonically with horizon length. We challenge that assumption in this work.

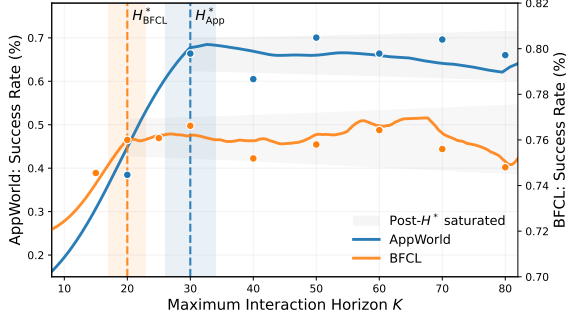
We present evidence that horizon scaling exhibits *diminishing marginal returns*. Figure 1a shows results from fixed-horizon sweep experiments on AppWorld (Trivedi et al., 2024): success rate increases with the interaction budget K in the under-capacity regime, then *plateaus* around a threshold we term the *effective interaction frontier* H^* . Beyond this point, additional budget produces no systematic improvement while computational cost—proportional to trajectory length—continues to grow linearly.

This saturation phenomenon has been observed independently in prior work. Xi et al. (2025) report that RL with a large interaction budget “quickly collapses,” and that beginning with a large number of interaction turns “leads the model into redundant reasoning and unproductive actions.” Liu et al. (2025) find that a ReAct agent “saturates at a budget of 100” tool calls and “fails to utilize additional tool budget, reaching a performance ceiling.” Shen et al. (2025) observe that a fixed long horizon ($h = 30$) leads the agent to expend steps unproductively early in training, degrading performance relative to a shorter horizon.

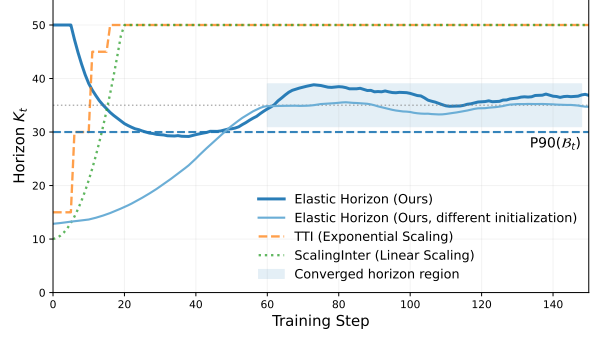
These observations converge on a common insight: there exists an *effective interaction frontier*

*Equal contribution.

†Corresponding authors.



(a) Fixed-horizon training on AppWorld: success rate improves with horizon K until reaching a plateau around the effective frontier H^* (shaded region), beyond which additional budget yields diminishing returns while cost grows linearly.



(b) Horizon evolution during training: open-loop schedules (ScalingInter, TTI) increase monotonically toward $K_{\max} = 50$, whereas Elastic Horizon stabilizes the horizon inside the saturation band from both low ($K_0 = 10$) and high ($K_0 = 50$) initializations.

Figure 1: Interaction horizon saturation and Elastic Horizon.

H^* , jointly determined by task complexity and agent capability, beyond which additional interaction steps yield diminishing returns. This motivates a fundamental question: **can we automatically detect and adapt to this frontier?**

We answer affirmatively with **Elastic Horizon**, a closed-loop horizon controller that automatically discovers and tracks H^* during training.¹ Unlike open-loop schedules that increase the horizon based on training progress, Elastic Horizon adjusts based on the agent’s *demonstrated capability*, specifically the lengths of trajectories that successfully complete tasks. It estimates the agent’s competence boundary using the 90th percentile of successful trajectory lengths, a statistic that captures near-maximal capability while filtering outlier noise from inefficient explorations.

The controller is *bidirectional*: expanding when the agent demonstrates capability growth, and contracting when approaching the task’s complexity ceiling (Figure 1b). This enables automatic convergence to H^* from arbitrary initializations. Open-loop schedules can only increase the horizon, so they cannot converge from an over-capacity start.

Elastic Horizon complements test-time budget-aware methods like BATS (Liu et al., 2025). While BATS optimizes deployment performance by injecting budget awareness into prompts, Elastic Horizon optimizes training efficiency by adapting the horizon to demonstrated capability. The two approaches target different phases and can be combined.

Our contributions are:

- **Empirical discovery.** We document interaction horizon saturation on AppWorld (Trivedi et al., 2024) and BFCL v3 (Patil et al., 2025), revealing clear plateaus beyond a task-dependent threshold H^* (Section 5.2).
- **Methodology.** We propose Elastic Horizon, the first closed-loop horizon controller for agentic RL. It adapts via percentile-based boundary estimation with EMA smoothing, requiring no manual schedule (Section 4).
- **Mechanism validation.** Elastic Horizon stabilizes the horizon inside the saturation band from both high and low initializations, validating its role as an automatic frontier detector (Section 5.3).
- **Practical impact.** Elastic Horizon attains the best results across both 7B and 14B backbones while reducing per-step trajectory tokens by up to 25% on AppWorld (Section 5.4).

Together, these contributions shift the focus from *how to scale* interaction horizons to *when to stop scaling*.

2 Related Work

2.1 Multi-Turn Agent Training and Horizon Scaling

Training LLM agents for long-horizon tasks presents challenges in stability and sample efficiency. Xi et al. (2025) introduce AgentGym-RL and propose ScalingInter-RL—a curriculum that progressively extends interaction horizons during training—finding that fixed large horizons cause training instability while gradual expansion im-

¹Code and training configurations are available at <https://github.com/junjie-meng/ElasticHorizon>.

proves performance. Shen et al. (2025) demonstrate that scaling interaction steps outperforms scaling per-step reasoning tokens under fixed compute budgets, advocating a multiplicative horizon schedule. Both methods employ *open-loop* schedules where the horizon increases according to pre-determined functions of training steps. Elastic Horizon differs by using *closed-loop* control: the horizon adapts based on observed trajectory statistics, enabling automatic convergence to the task’s effective frontier without manual schedule design.

Several works address long-horizon challenges through complementary approaches. Zhou et al. (2024) propose ArCHer, combining hierarchical off-policy learning with on-policy token-level optimization. Wang et al. (2025) document the “Echo Trap” phenomenon and propose variance-based filtering for stabilization. Qi et al. (2025) introduce self-evolving task curricula, and Chen et al. (2025) develop memory-efficient PPO variants. These methods address orthogonal aspects of long-horizon training and can be combined with Elastic Horizon’s adaptive horizon control.

2.2 Budget-Aware Agents

A parallel thread addresses resource allocation at inference time. Liu et al. (2025) propose BATS, which injects remaining budget information into prompts and adapts planning strategies based on available resources. Snell et al. (2025) establish that compute-optimal test-time strategies should adaptively allocate resources based on prompt difficulty. Paglieri et al. (2025) learn *when* to invoke planning versus direct action. Han et al. (2025) dynamically estimate token budgets based on reasoning complexity.

These methods focus on *test-time* resource optimization with explicit budget signals. Elastic Horizon operates during *training* using implicit capability signals (trajectory length distributions). The approaches are complementary: Elastic Horizon optimizes training efficiency, while methods like BATS optimize deployment performance.

2.3 Curriculum Learning

Curriculum learning (Bengio et al., 2009) demonstrates that presenting examples from easy to hard improves generalization. Self-paced learning (Kumar et al., 2010) extends this by determining difficulty from the learner’s current loss, making the curriculum data-driven. Jiang et al. (2015) unify these approaches, combining prior difficulty knowl-

Table 1: Positioning of Elastic Horizon.[†]

Method	Stage	Control	Signal
ScalingInter-RL	Train	Open-loop	Training step
TTI	Train	Open-loop	Training step
BATS	Test	Closed-loop	Budget count
Self-Paced	Train	Closed-loop	Loss value
Elastic (Ours)	Train	Closed-loop	Traj. length

[†]Elastic Horizon uses the 90th percentile of successful trajectory lengths; see Section 4.3.

edge with dynamic learner feedback. In reinforcement learning, curriculum methods have been applied to task selection (Narvekar et al., 2020), goal generation (Florensa et al., 2018), and start state distributions (Florensa et al., 2017).

Elastic Horizon contributes a novel competence signal—the upper percentile of successful trajectory lengths—applied to episode horizon length, a previously underexplored curriculum dimension. This signal directly measures demonstrated capability to complete long-horizon tasks, making it particularly suited for agentic RL where trajectory length correlates with task complexity.

2.4 Scaling Laws

Scaling laws research establishes that optimal resource allocation depends on model capacity and task demands. Kaplan et al. (2020) derive power-law relationships between performance and compute. Hoffmann et al. (2022) demonstrate that model size and training tokens should scale proportionally for compute-optimal training. Hilton et al. (2023) extend these findings to RL, showing that varying the task’s horizon length changes the coefficient but not the exponent of the compute-optimal scaling relationship. These principles support Elastic Horizon’s premise: interaction horizons should adapt to agent competence rather than follow fixed maximal allocations.

Table 1 summarizes Elastic Horizon’s positioning among related methods.

3 Preliminaries

3.1 Multi-Turn Agentic Tasks as POMDPs

We formalize multi-turn agentic tasks as episodic decision problems with sparse terminal rewards. Following standard practice in agentic RL (Xi et al., 2025; Wang et al., 2025), we adopt a simplified Partially Observable Markov Decision Process (POMDP) formulation:

$$\mathcal{M} = (\mathcal{U}, \mathcal{S}, \mathcal{A}, \mathcal{O}, T, R), \quad (1)$$

where $\mathcal{U}$ is the instruction space specifying task goals, $\mathcal{S}$ is the state space (only partially observable to the agent), $\mathcal{A}$ is the action space (e.g., tool calls, API requests), $\mathcal{O}$ is the observation space (e.g., API responses, environment feedback), $T : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ is the state transition function, and $R : \mathcal{S} \rightarrow \{0, 1\}$ is a sparse binary terminal reward indicating task success.

The agent’s policy π_θ is parameterized by a large language model with parameters θ . Given an instruction $u \in \mathcal{U}$ and the interaction history $h_k = (a_1, o_1, \dots, a_{k-1}, o_{k-1})$, the policy outputs a distribution over actions: $\pi_\theta(a_k|u, h_k)$. We denote a complete trajectory as $\tau = (u, a_1, o_1, \dots, a_L, o_L)$, where $L(\tau)$ denotes the trajectory length (number of interaction turns).

3.2 Success Rate under Horizon Constraints

Let $r(\tau) \in \{0, 1\}$ denote the terminal reward of trajectory τ , where $r(\tau) = 1$ indicates successful task completion. During training, we impose an *interaction budget* (or *horizon constraint*) K : a trajectory is forcibly terminated if it reaches K steps without task completion, receiving $r(\tau) = 0$.

For a task distribution $\mathcal{T}$, policy π_θ , and horizon constraint K , we define the *success rate* as:

$$\text{SR}(K; \pi_\theta, \mathcal{T}) = \mathbb{E}_{u \sim \mathcal{T}, \tau \sim \pi_\theta(\cdot|u, K)} [r(\tau)], \quad (2)$$

where the notation $\tau \sim \pi_\theta(\cdot|u, K)$ indicates that the trajectory is sampled under horizon constraint K . The horizon constraint thus defines the maximum number of interaction turns the agent may use per episode, directly affecting both task completion probability and computational cost.

3.3 Horizon Scheduling in Curriculum Learning

Existing curriculum-based methods adjust the horizon constraint K_t at training step t according to predetermined schedules:

Linear schedule (Xi et al., 2025): The horizon increases by a fixed amount per training step:

$$K_t = \min(K_{\min} + \delta \cdot t, K_{\max}), \quad (3)$$

where $\delta > 0$ is the increment rate, and $K_{\min}, K_{\max}$ are the lower and upper bounds.

Multiplicative schedule (Shen et al., 2025): The horizon is set to a growing integer multiple of $K_{\min}$ at stage boundaries:

$$K_t = \min(K_{\min} \cdot (\lfloor t/T_{\text{stage}} \rfloor + 1), K_{\max}), \quad (4)$$

where T_{stage} is the stage duration. Shen et al. (2025) write this as $h_i = \text{clip}(h_{\min} \cdot i, h_{\max})$ for iteration i , and adopt it over the additive alternative $h_i = \text{clip}(h_{\min} + i, h_{\max})$.

Both schedules are *open-loop*: K_t depends solely on the training step t and is independent of the agent’s actual performance or the task distribution’s complexity. In contrast, our method adapts K_t based on observed trajectory statistics, forming a *closed-loop* controller.

A complete notation summary is provided in Appendix A.

4 Method

We define the *effective interaction frontier* that motivates our design (Section 4.1), derive principled design requirements for a closed-loop controller (Section 4.2), present the capability boundary estimation mechanism (Section 4.3), and provide the complete Elastic Horizon algorithm (Section 4.4).

4.1 The Effective Interaction Frontier

Open-loop horizon schedules implicitly assume that performance improves monotonically with interaction budget. The fixed-horizon sweeps in Section 5.2 (Figure 3) refute this assumption: success rate plateaus once the interaction budget exceeds a task-dependent threshold, after which additional budget yields no measurable gain in success rate while training compute continues to grow linearly.

We call this threshold the *effective interaction frontier* and denote it $H^*(\mathcal{T}, \theta)$, where $\mathcal{T}$ is the task distribution and θ the policy parameters. The frontier is the operationally relevant boundary: training with $K > H^*$ pays the linear cost of additional steps without raising success rate, while training with $K < H^*$ caps the agent below its achievable performance. We treat H^* as an empirical quantity defined by the plateau in Figure 3 rather than introducing further formalism, and design our method to track it.

The frontier is *dynamic*: it depends on both $\mathcal{T}$ and the evolving θ . As the agent improves, H^* may shift—a more capable agent can solve harder tasks that require longer interaction sequences. This time-varying property is what makes a fixed K inadequate and motivates a *closed-loop* controller that re-estimates H^* throughout training.

We do not assume that H^* is directly observable. Instead, we use the agent’s successful-trajectory length distribution as a behavioral surrogate (Sec-

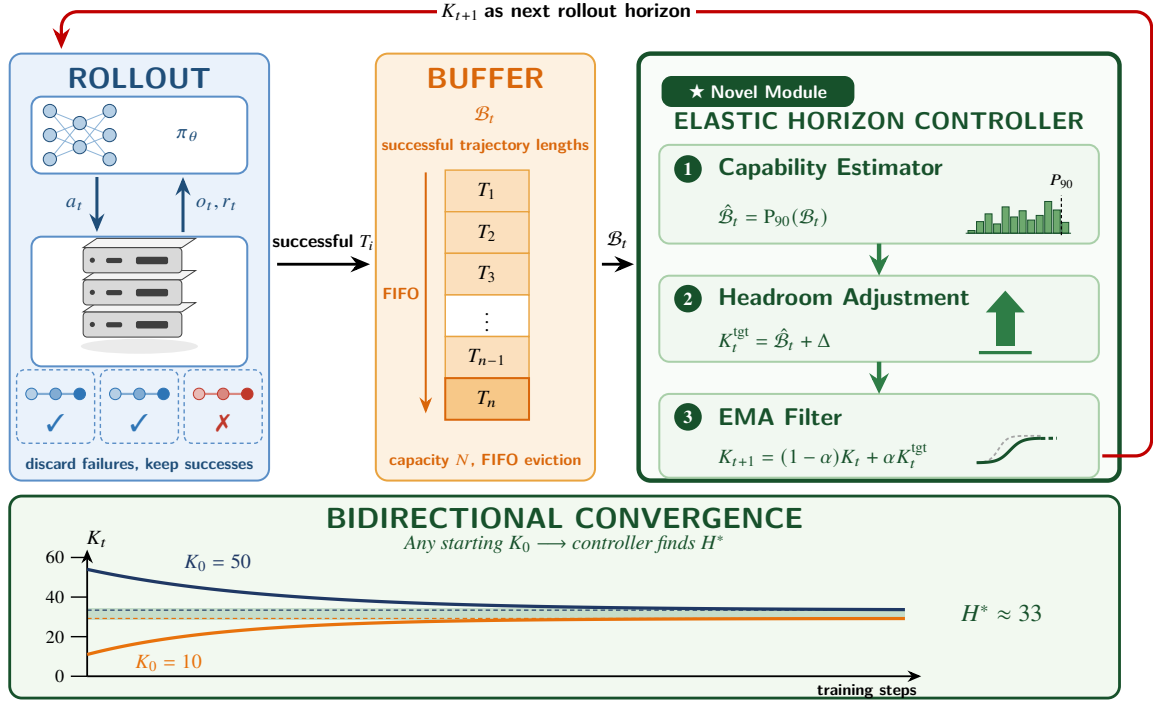


Figure 2: **Elastic Horizon overview.** Successful trajectories from ROLLOUT populate the FIFO BUFFER $\mathcal{B}_t$ of trajectory lengths. The ELASTIC HORIZON CONTROLLER (right) maps the buffer to the next horizon in three stages: a percentile *capability estimator* $\hat{B}_t = P_{90}(\mathcal{B}_t)$, an additive *headroom adjustment* $K_t^{\text{tgt}} = \hat{B}_t + \Delta$, and an *EMA filter* $K_{t+1} = (1 - \alpha)K_t + \alpha K_t^{\text{tgt}}$. The output K_{t+1} feeds back as the next rollout’s horizon (red arrow), forming a closed loop. The bottom panel shows the controller’s key property: *bidirectional convergence*—from any initialization K_0 , the horizon settles around the effective frontier H^* .

tion 4.3), and validate empirically (Section 5.3) that the controller’s converged horizon lies within the saturation band identified in Section 5.2.

4.2 Design Requirements for Closed-Loop Control

Given the goal of tracking H^* , we identify three design requirements:

R1: Capability-driven signal. The controller should adjust the horizon based on the agent’s demonstrated capability, not training progress. Open-loop schedules violate this by conditioning on step t rather than performance.

R2: Robust estimation. The capability signal must be robust to trajectory-level noise (e.g., anomalously long trajectories from inefficient exploration) while remaining sensitive to genuine capability changes.

R3: Bidirectional adaptation. The controller must expand the horizon when capability grows and contract when approaching saturation. Monotonic-only schedules cannot converge to H^* from above.

Elastic Horizon satisfies these requirements through: (R1) using successful trajectory lengths

as the capability signal; (R2) employing percentile-based estimation with smoothing; (R3) allowing both expansion and contraction based on the signal.

4.3 Capability Boundary Estimation

The core mechanism of Elastic Horizon is estimating the agent’s current capability boundary from trajectory data. We maintain a buffer $\mathcal{B}$ of successful trajectory lengths and compute an upper percentile as the boundary estimate.

Why successful trajectories? Successful trajectories directly measure the agent’s ability to complete tasks within a given number of steps. The distribution of successful trajectory lengths F_L encodes how much interaction budget the agent *actually needs* to solve tasks it can solve. This provides a grounded capability signal, unlike training loss or reward statistics which conflate capability with task difficulty.

Percentile selection. We use the **90th percentile (P90)** of successful trajectory lengths:

$$\hat{B}_t = \hat{F}_L^{-1}(0.9) = P_{90}(\mathcal{B}_t), \quad (5)$$

where $\hat{F}_L(x)$ is the empirical cumulative distribution function (CDF) of trajectory lengths in buffer $\mathcal{B}_t$.

This choice balances two competing objectives:

Sensitivity: The estimate should capture near-maximal capability, providing sufficient horizon for harder task instances. The mean $\mathbb{E}[L]$ underestimates this, as it reflects average behavior rather than capability limits.

Robustness: The estimate should filter outliers from inefficient exploration or random walks. The maximum $\max_i L_i$ is sensitive to a single anomalously long trajectory.

P90 achieves both: it captures the capability frontier (90% of successful trajectories complete within this length) while filtering the top 10% of potential outliers. From an order statistics perspective, high quantiles provide robust location estimates for heavy-tailed distributions. Agent trajectory lengths are commonly heavy-tailed, as we verify empirically in Appendix E.

Exploration headroom. To encourage attempting tasks slightly beyond current capability, we add a fixed headroom Δ :

$$K_t^{\text{target}} = \hat{B}_t + \Delta. \quad (6)$$

The headroom provides slack for exploring longer trajectories necessary for harder tasks, preventing premature convergence to a suboptimal frontier. This parallels the “zone of proximal development” concept in curriculum learning (Bengio et al., 2009): the agent should be challenged slightly beyond demonstrated competence.

EMA smoothing. Raw percentile estimates fluctuate due to sampling variance. We apply exponential moving average (EMA) smoothing to the horizon update:

$$K_{t+1} = (1-\alpha) \cdot K_t + \alpha \cdot \text{clip}(K_t^{\text{target}}, K_{\min}, K_{\max}), \quad (7)$$

where $\alpha \in (0, 1)$ controls adaptation speed. Small α yields stable but slow adaptation; large α yields responsive but potentially oscillatory behavior.

4.4 The Elastic Horizon Algorithm

Algorithm 1 presents the complete training procedure integrating the above components.

Cold start handling. When successful samples are scarce ($|\mathcal{B}| < N_{\min}$), the P90 estimate is unreliable. The controller maintains the current horizon

Algorithm 1 Elastic Horizon Training

Require: Policy π_{θ_0} , task distribution $\mathcal{T}$

Require: Initial horizon K_0 , bounds $[K_{\min}, K_{\max}]$

Require: EMA coefficient α , headroom Δ , buffer size N

```

1: Initialize buffer  $\mathcal{B} \leftarrow \emptyset$ , horizon  $K \leftarrow K_0$ 
2: for training step  $t = 1, 2, \dots, T$  do
3:   // Trajectory collection under current horizon
4:   Sample trajectories  $\{\tau_i\}_{i=1}^M$  with budget  $K$ 
5:   // Policy update
6:    $\theta_t \leftarrow \text{RLUPDATE}(\theta_{t-1}, \{\tau_i\})$ 
7:   // Buffer update (FIFO)
8:   for each  $\tau_i$  with  $r(\tau_i) = 1$  do
9:      $\mathcal{B}.\text{APPEND}(L(\tau_i))$ 
10:    if  $|\mathcal{B}| > N$  then  $\mathcal{B}.\text{POPFIRST}()$ 
11:    end if
12:  end for
13:  // Closed-loop horizon update
14:  if  $|\mathcal{B}| \geq N_{\min}$  then
15:     $\hat{B} \leftarrow \text{P90}(\mathcal{B})$   $\triangleright$  Capability estimate
16:     $K^{\text{raw}} \leftarrow \text{clip}(\hat{B} + \Delta, K_{\min}, K_{\max})$ 
17:     $K \leftarrow (1 - \alpha) \cdot K + \alpha \cdot K^{\text{raw}}$   $\triangleright$  EMA
18:  end if
19: end for
```

Table 2: Comparison of horizon scheduling approaches.

	ScalingInter	TTI	Elastic (Ours)
Control	Open-loop	Open-loop	Closed-loop
Signal	Step t	Step t	P90 of L_{succ}
Direction	$\uparrow$ only	$\uparrow$ only	Bidirectional
Converges to	$K_{\max}$	$K_{\max}$	$\approx H^*$
Hparams	δ , stages	$K_{\min}, T_{\text{stg}}$	α, Δ

until sufficient data accumulates. This conservative strategy prevents erratic behavior during early training when exploration dominates.

Computational overhead. Elastic Horizon adds negligible cost: maintaining a buffer of integers, computing a percentile, and performing EMA update require $O(N)$ operations per step, which is insignificant compared to trajectory sampling and gradient computation.

Comparison with open-loop schedules. Table 2 summarizes key differences. Open-loop methods depend solely on training step t and increase monotonically to $K_{\max}$. Elastic Horizon depends on agent performance and adapts bidirectionally, automatically converging toward H^* .

5 Experiments

We design experiments to answer three research questions: **RQ1**: Does interaction horizon saturation exist? **RQ2**: Can Elastic Horizon automatically discover the effective frontier? **RQ3**: How does Elastic Horizon compare to open-loop baselines?

5.1 Experimental Setup

Environments. We evaluate on two challenging multi-turn agent benchmarks:

AppWorld (Trivedi et al., 2024) is a high-fidelity execution environment simulating 9 daily applications (Amazon, Venmo, Spotify, Gmail, etc.) with 457 APIs. Tasks require agents to coordinate across multiple apps through API calls. We report task goal completion rate.

BFCL v3 (Patil et al., 2025) (Berkeley Function Calling Leaderboard) evaluates multi-turn function calling across 8 API domains. The multi-turn split contains 1,000 test cases requiring stateful interactions. We report overall success rate.

Both benchmarks use binary sparse rewards and enforce a maximum trajectory length of 50 steps.

Baselines. We compare against: **GRPO (Fixed K)**: Group Relative Policy Optimization (Shao et al., 2024) with static horizon constraints ($K = 15$ and $K = 50$). **TTI** (Shen et al., 2025): Multiplicative horizon schedule ($15 \rightarrow 20 \rightarrow 30 \rightarrow 50$). **ScalingInter** (Xi et al., 2025): Linear horizon schedule from $K_{\min} = 10$ to $K_{\max} = 50$.

Training details. We use GRPO with KL coefficient zero, training for 200 steps with batch size 16 and learning rate 1×10^{-6} on both Qwen2.5-7B and Qwen2.5-14B. Our training pipeline is built on the AgentEvolver agent training framework (Zhai et al., 2025). For Elastic Horizon: $K_0 = 15$, $\alpha = 0.1$, $\Delta = 10$, buffer size $N = 100$. We report mean@8 (average over 8 rollouts). Full details in Appendix D.

5.2 RQ1: Does Horizon Saturation Exist?

To verify interaction horizon saturation, we train separate models with fixed horizon constraints, sweeping K from 10 up to 80 on both benchmarks.

Figure 3 reveals a consistent saturation pattern on both benchmarks. In the *under-capacity regime* ($K < H^*$), reward increases substantially with horizon length: AppWorld 7B climbs from $\sim 17\%$ at $K=10$ to $\sim 60\%$ at $K=30$. In the *saturation*

regime ($K \approx H^*$), reward plateaus—runs with $K \in [30, 60]$ converge to the same band on AppWorld, and $K \in [20, 50]$ converge on BFCL—despite a $2\times$ range of per-step cost. In the *noise-dominated regime* ($K > H^*$), additional budget yields no systematic improvement.

The saturation threshold differs between benchmarks: AppWorld, with complex multi-app coordination, exhibits a higher H^* than BFCL’s function-calling tasks. This validates that H^* is task-dependent and underscores the need for adaptive horizon control.

5.3 RQ2: Automatic Frontier Discovery

We test Elastic Horizon’s ability to adapt the horizon based on agent capability by running from two extreme initializations: $K_0 = 10$ (under-capacity) and $K_0 = 50$ (over-capacity). Figure 1b (right panel of Figure 1) shows the horizon trajectories. Open-loop baselines (ScalingInter, TTI) increase monotonically to $K_{\max} = 50$ regardless of agent behavior. Elastic Horizon behaves differently: from $K_0 = 10$, the horizon expands as the buffer fills with longer successful trajectories; from $K_0 = 50$, it contracts once the P90 estimate stabilizes. In both cases, the horizon stabilizes inside the saturation band identified in RQ1 (Figure 3). The exact stable value differs between runs (since it depends on hyperparameters α and Δ), but in neither case does the horizon reach $K_{\max}$.

We interpret this as evidence that Elastic Horizon adapts the horizon based on the agent’s demonstrated trajectory length distribution rather than training step, which is the property that distinguishes it from open-loop schedules.

5.4 RQ3: Comparison with Baselines

Table 3 summarizes the main results.

Static horizon dilemma. GRPO ($K=15$) vs. GRPO ($K=50$) illustrates the challenge of fixed-horizon training. On AppWorld, longer horizons help (complex multi-app tasks benefit from extended interaction); on BFCL they hurt, with training instability (marked *), confirming that exceeding H^* degrades learning efficiency.

Open-loop limitations. TTI and ScalingInter give inconsistent results across benchmarks: both reach $K_{\max}$ regardless of task complexity, leaving them unable to adapt when the optimal horizon differs across domains.

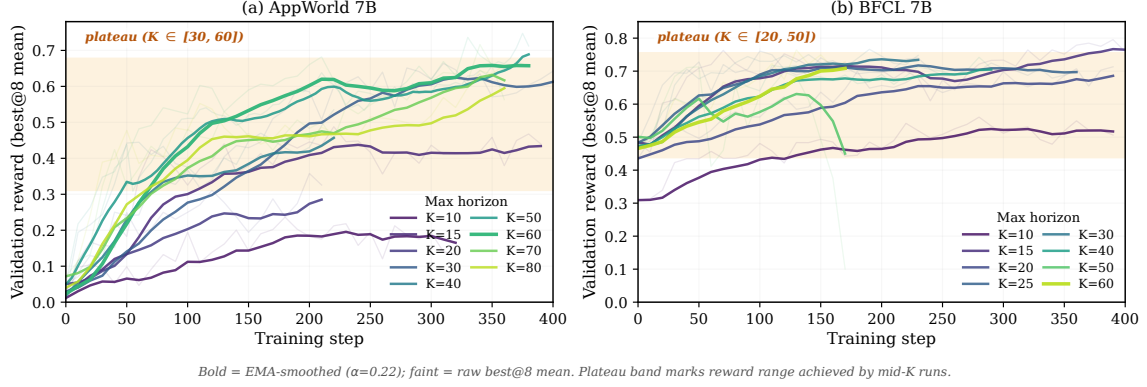


Figure 3: **Training dynamics under fixed horizons.** Validation reward (best@8 mean) versus training step for fixed-horizon GRPO runs on AppWorld 7B (a) and BFCL 7B (b). Curves are colored by K on a viridis scale; bold lines are EMA-smoothed ($\alpha=0.22$), faint lines are raw measurements. On both benchmarks, performance saturates within a benchmark-specific plateau band (orange shading): $K \in [30, 60]$ on AppWorld and $K \in [20, 50]$ on BFCL. Beyond the plateau, additional horizon budget produces no measurable gain in reward despite linearly higher per-step cost.

Table 3: Performance comparison. Upper: zero-shot backbone models. Lower: training with Qwen2.5-7B and Qwen2.5-14B. *: training instability. Best in **bold**.

Method	AppWorld (%)		BFCL (%)	
<i>Zero-shot Baselines</i>				
Qwen2.5-7B		2.08		30.12
Qwen2.5-14B		18.00		41.62
Qwen2.5-32B		39.06		49.63
Qwen3-235B-A22B		33.85		60.88
<i>Training (7B)</i>	Mean@8	Best@8	Mean@8	Best@8
GRPO ($K = 15$)	21.49	42.40	66.87	71.62
GRPO ($K = 50$)	31.14	49.20	60.37*	68.32*
TTI	37.28	46.14	55.75	71.22
ScalingInter	36.18	55.10	69.12	73.93
Elastic Horizon	39.47	56.87	71.62	80.45
<i>Training (14B)</i>	Mean@8	Best@8	Mean@8	Best@8
GRPO ($K = 15$)	56.57	71.16	68.80	74.13
GRPO ($K = 50$)	65.78	80.63	64.12	73.61
TTI	60.08	77.73	70.54	76.35
ScalingInter	67.54	82.54	70.17	78.85
Elastic Horizon	77.85	90.42	73.62	80.49

Elastic Horizon robustness. Elastic Horizon achieves the best results on both benchmarks at both 7B and 14B (e.g., AppWorld Best@8=90.42% at 14B). The trained 7B model surpasses zero-shot Qwen2.5-32B (39.06%) and Qwen3-235B-A22B (33.85%) on AppWorld.

Sample efficiency. Elastic Horizon saves tokens at two regimes (Figure 4): at mid-training (steps 100–170) when GRPO peaks, EH has already converged and saves 25% per-step tokens; at convergence (step ≥ 200), EH still uses 11% fewer per-step tokens than the worst baseline. These add up

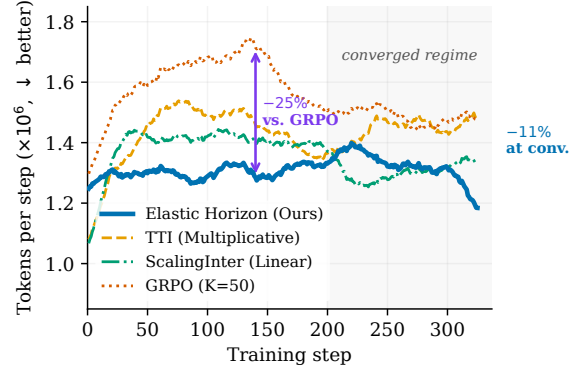


Figure 4: **Per-step token consumption on AppWorld (14B).** Elastic Horizon consumes -25% vs. GRPO at the mid-training peak (step ~ 140) and -11% vs. the worst baseline at convergence (step ≥ 200). Cumulative-token and BFCL counterparts in Appendix F.4.

to a 28% cumulative-token saving at SR=50% on AppWorld 7B (Appendix F.4). Under tight compute budgets, the mid-training saving is the more consequential.

6 Conclusion

We introduced *Elastic Horizon*, a closed-loop controller that sets the interaction budget of an agentic RL run from the agent’s own demonstrated behavior, estimating the effective interaction frontier H^* from the 90th percentile of successful trajectory lengths. Three findings emerge. First, fixed-horizon training saturates beyond a task-dependent H^* : on AppWorld and BFCL, a twofold range of interaction budgets converges to the same reward band while per-step cost keeps growing. Sec-

ond, because the controller reads capability rather than training step, it both expands and contracts the horizon, and settles inside the saturation band from under- and over-capacity initializations alike, behavior that monotonically increasing schedules cannot produce. Third, this costs nothing in quality: Elastic Horizon attains the best success rates across 7B and 14B backbones on both benchmarks while cutting per-step trajectory tokens by up to 25%.

The broader point is that the interaction budget need not be a hyperparameter. Open-loop curricula ask practitioners to commit in advance to a growth rate and a maximum horizon, quantities that depend on the agent and the environment and are therefore unknown before training begins. Elastic Horizon replaces that commitment with a measurement, so that how far to scale the horizon—and where to stop—is determined by the training run itself. We expect the same treatment, allocating from demonstrated capability in both directions, to be applicable to other budgets in agent training beyond the interaction horizon.

Limitations

We discuss several limitations of our work that suggest directions for future research.

Cold start with sparse success. Elastic Horizon relies on successful trajectory statistics for capability estimation. In early training or on extremely difficult tasks, successful trajectories may be rare, leading to unreliable P90 estimates. Our current mitigation—maintaining the initial horizon until a minimum buffer is reached—is conservative but may delay adaptation. Future work could explore alternative signals such as partial progress indicators or trajectory length distributions from all rollouts (not just successful ones).

Single global horizon. We adjust a single global horizon K_t applied uniformly across all task instances. For heterogeneous task distributions with wide difficulty variance, this may be suboptimal: easy tasks receive unnecessarily long horizons while hard tasks may still be under-budgeted. Instance-level or difficulty-conditioned horizon allocation could improve efficiency further, though at the cost of additional complexity.

Heuristic nature of P90. The choice of the 90th percentile is empirically motivated and supported by ablation studies, but lacks a first-principles derivation. While we provide intuitions from order

statistics and curriculum learning theory, a more rigorous theoretical foundation for optimal percentile selection remains an open question. The ablation results suggest that performance is relatively robust across the P75–P95 range, but the optimal choice may be task-dependent.

Proxy efficiency metric. We use cumulative trajectory tokens as a proxy for training compute. While more informative than raw interaction counts, this metric does not capture all cost factors such as environment latency, parallel sampling efficiency, or accelerator utilization. Actual wall-clock time savings may differ from token-based estimates.

Ethical Considerations

Our proposed method, Elastic Horizon, explicitly addresses the computational overhead associated with long-horizon agentic tasks. By automatically detecting the effective interaction frontier, our approach reduces per-step trajectory tokens by up to 25% during the most expensive phase of training and 11% at convergence on AppWorld, compared to fixed-horizon baselines. This contributes to the goals of “Green AI” by lowering the energy consumption and carbon footprint required to train capable agents. By demonstrating that smaller models (e.g., 7B parameters) can achieve high performance through adaptive horizon control, our work also helps make agent research more accessible to institutions with limited computational resources.

While we focus on training efficiency, we acknowledge that more capable LLM agents carry inherent risks of misuse. Our method does not introduce specific new capabilities that would uniquely exacerbate these risks beyond the general progress of the field; it is a training-efficiency technique applicable to any agentic RL setup. We use only publicly released benchmarks (AppWorld, BFCL) and base models (Qwen2.5), and our experiments do not involve human subjects or sensitive data.

References

- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. [Curriculum learning](#). In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pages 41–48. ACM.
- Kevin Chen, Marco Cusumano-Towner, Brody Huval, Aleksei Petrenko, Jackson Hamburger, Vladlen

- Koltun, and Philipp Krähenbühl. 2025. [Reinforcement learning for long-horizon interactive LLM agents](#). *Preprint*, arXiv:2502.01600.
- Carlos Florensa, David Held, Xinyang Geng, and Pieter Abbeel. 2018. [Automatic goal generation for reinforcement learning agents](#). In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 1515–1528. PMLR.
- Carlos Florensa, David Held, Markus Wulfmeier, Michael Zhang, and Pieter Abbeel. 2017. [Reverse curriculum generation for reinforcement learning](#). In *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, volume 78 of *Proceedings of Machine Learning Research*, pages 482–495. PMLR.
- Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. 2025. [Token-budget-aware LLM reasoning](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24842–24855, Vienna, Austria. Association for Computational Linguistics.
- Jacob Hilton, Jie Tang, and John Schulman. 2023. [Scaling laws for single-agent reinforcement learning](#). *Preprint*, arXiv:2301.13442.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Thomas Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karén Simonyan, Erich Elsen, and 3 others. 2022. [An empirical analysis of compute-optimal large language model training](#). In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 30016–30030. Curran Associates, Inc.
- Lu Jiang, Deyu Meng, Qian Zhao, Shiguang Shan, and Alexander G. Hauptmann. 2015. [Self-paced curriculum learning](#). In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 2694–2700.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. [Scaling laws for neural language models](#). *Preprint*, arXiv:2001.08361.
- M. Pawan Kumar, Benjamin Packer, and Daphne Koller. 2010. [Self-paced learning for latent variable models](#). In *Advances in Neural Information Processing Systems 23 (NeurIPS)*, pages 1189–1197.
- Tengxiao Liu, Zifeng Wang, Jin Miao, I-Hung Hsu, Jun Yan, Jiefeng Chen, Rujun Han, Fangyuan Xu, Yanfei Chen, Ke Jiang, Samira Daruki, Yi Liang, William Yang Wang, Tomas Pfister, and Chen-Yu Lee. 2025. [Budget-aware tool use enables effective agent scaling](#). *Preprint*, arXiv:2511.17006. Accepted to COLM 2026.
- Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. 2020. [Curriculum learning for reinforcement learning domains: A framework and survey](#). *Journal of Machine Learning Research*, 21(181):1–50.
- Davide Paglieri, Bartłomiej Cupiał, Jonathan Cook, Ulyana Piterbarg, Jens Tuyls, Edward Grefenstette, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. 2025. [Learning when to plan: Efficiently allocating test-time compute for LLM agents](#). *Preprint*, arXiv:2509.03581.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. [The Berkeley function calling leaderboard \(BFCL\): From tool use to agentic evaluation of large language models](#). In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR.
- Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Jiadai Sun, Xinyue Yang, Yu Yang, Shuntian Yao, Wei Xu, Jie Tang, and Yuxiao Dong. 2025. [WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning](#). In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. [DeepSeekMath: Pushing the limits of mathematical reasoning in open language models](#). *Preprint*, arXiv:2402.03300.
- Junhong Shen, Hao Bai, Lunjun Zhang, Yifei Zhou, Amrith Setlur, Peter Tong, Diego Caples, Nan Jiang, Tong Zhang, Ameet Talwalkar, and Aviral Kumar. 2025. [Thinking vs. Doing: Improving agent reasoning by scaling test-time interaction](#). In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, pages 169640–169681. Curran Associates, Inc.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: Language agents with verbal reinforcement learning](#). In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pages 8634–8652. Curran Associates, Inc.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2025. [Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning](#). In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*.
- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjana Balasubramanian. 2024. [AppWorld: A controllable world of apps and people for benchmarking interactive coding agents](#).

In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, Bangkok, Thailand. Association for Computational Linguistics.

Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. 2025. [RA-GEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning](#). *Preprint*, arXiv:2504.20073.

Zhiheng Xi, Jixuan Huang, Chenyang Liao, Baodai Huang, Honglin Guo, Jiaqi Liu, Rui Zheng, Junjie Ye, Jiazheng Zhang, Wenxiang Chen, Wei He, Yiwen Ding, Guanyu Li, Zehui Chen, Zhengyin Du, Xuesong Yao, Yufei Xu, Jiecao Chen, Tao Gui, and 4 others. 2025. [AgentGym-RL: Training LLM agents for long-horizon decision making through multi-turn reinforcement learning](#). *Preprint*, arXiv:2509.08755.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [ReAct: Synergizing reasoning and acting in language models](#). In *The Eleventh International Conference on Learning Representations (ICLR 2023)*.

Yunpeng Zhai, Shuchang Tao, Cheng Chen, Anni Zou, Ziqian Chen, Qingxu Fu, Shinji Mai, Li Yu, Jiaji Deng, Zouying Cao, Zhaoyang Liu, Bolin Ding, and Jingren Zhou. 2025. [AgentEvolver: Towards efficient self-evolving agent system](#). *Preprint*, arXiv:2511.10395.

Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, and Aviral Kumar. 2024. [ArCHer: Training language model agents via hierarchical multi-turn RL](#). In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 62178–62209. PMLR.

A Notation Summary

Table 4 provides a complete summary of notation used throughout the paper.

B Theoretical Analysis

This appendix collects formal discussion of the design choices in Elastic Horizon. We do not claim a full theoretical analysis: the controller is fundamentally a closed-loop heuristic whose primary justification is empirical (Section 5). What we make explicit here are (i) the statistical motivation for using the 90th percentile of successful trajectory lengths as a capability boundary estimator (Section B.1); and (ii) the stability behavior we rely on for the EMA update, together with the limitations of treating it as a convergence result (Section B.2).

Table 4: Complete notation summary.

Symbol	Description	Defined in
<i>POMDP Formulation</i>		
$\mathcal{M}$	POMDP tuple	Sec. 3.1
$\mathcal{U}$	Instruction space	Sec. 3.1
$\mathcal{S}$	State space	Sec. 3.1
$\mathcal{A}$	Action space	Sec. 3.1
$\mathcal{O}$	Observation space	Sec. 3.1
T	Transition function	Sec. 3.1
R	Reward function	Sec. 3.1
π_θ	Policy w/ params θ	Sec. 3.1
$\mathcal{T}$	Task distribution	Sec. 3.2
<i>Trajectory and Metrics</i>		
τ	Trajectory	Sec. 3.1
$L(\tau)$	Trajectory length (turns)	Sec. 3.1
$r(\tau)$	Terminal reward (1=succ, 0=fail)	Sec. 3.2
$\text{SR}(K; \pi_\theta, \mathcal{T})$	Success rate under horizon K	Sec. 3.2
<i>Horizon Control</i>		
K_t	Horizon at step t	Sec. 3.3
$K_{\min}$	Minimum horizon bound	Sec. 3.3
$K_{\max}$	Maximum horizon bound	Sec. 3.3
H^*	Effective frontier	Sec. 4.1
<i>Elastic Horizon</i>		
$\mathcal{B}$	Success buffer (trajectory lengths)	Sec. 4.4
$\hat{B}_t$	Capability boundary estimate	Sec. 4.3
$\text{P90}(\mathcal{B})$	90th percentile of $\mathcal{B}$	Sec. 4.3
F_L	CDF of success lengths	Sec. 4.3
α	EMA smoothing coefficient	Sec. 4.4
Δ	Headroom	Sec. 4.3
N	Buffer capacity	App. D
$N_{\min}$	Min. buffer for activation	App. D

B.1 Statistical Motivation for the P90 Estimator

The capability boundary estimator $\hat{B}_t = \text{P90}(\mathcal{B}_t)$ is motivated by two desiderata: it should reflect near-maximal demonstrated capability while remaining robust to occasional very long successful trajectories produced by inefficient exploration. Both desiderata follow from standard order-statistics intuition; we do not derive a closed-form relationship between $\hat{B}_t$ and the frontier H^* .

Heavy-tail robustness. Successful trajectory lengths are right-skewed in our experiments (Appendix E, Table 7). Under heavy tails, the sample maximum has variance

$$\text{Var}[\max_i L_i] = O(n^{2/\alpha-2}), \quad (8)$$

for Pareto tails with index α , which diverges for $\alpha < 2$. By contrast, sample quantiles in the body of the distribution—including the 90th percentile under standard regularity conditions—have variance $O(1/n)$. The P90 therefore provides a far more stable boundary estimate than the maximum when occasional very long successful trajectories occur.

Choice of P90 vs. other quantiles. The choice of the 90th percentile, rather than P75 or P95, is empirically motivated. Lower quantiles (mean, P50) underestimate near-maximal capability and lead to overly conservative horizons that truncate harder task instances; higher quantiles approach the maximum and lose the robustness benefit above. The ablation in Appendix F confirms that performance is relatively insensitive across the P75–P95 range, consistent with the order-statistics intuition that any high quantile gives a stable, near-maximal estimate. We do not claim that P90 is optimal in any formal sense.

Connection to the frontier. We do not establish a closed-form bound between $\hat{B}_t$ and H^* . The intended interpretation is qualitative: $\hat{B}_t$ captures the horizon at which 90% of the agent’s currently solvable tasks complete, so $\hat{B}_t + \Delta$ is a budget large enough to admit nearly all current successes plus some slack for exploration of harder instances. Whether this surrogate tracks the true frontier is an empirical question; the convergence experiments in Section 5.3, where the controller stabilizes near the same plateau identified by the fixed-horizon sweeps in Section 5.2, are the main evidence we offer.

B.2 Stability of the EMA Update

The horizon update rule

$$K_{t+1} = (1 - \alpha)K_t + \alpha K_t^{\text{raw}}, \quad (9)$$

$$K_t^{\text{raw}} = \text{clip}(\hat{B}_t + \Delta, K_{\min}, K_{\max}), \quad (10)$$

is a standard exponential moving average over the clipped target K_t^{raw} . Its stability is a property of the EMA filter rather than a novel result, but it is worth stating the form we rely on, together with what it does *not* say.

Geometric attraction toward a stationary target. Suppose the input sequence $\{K_t^{\text{raw}}\}$ converges to a fixed target $K^\dagger$. Subtracting $K^\dagger$ from both sides of the EMA update gives $|K_{t+1} - K^\dagger| \leq (1 - \alpha)|K_t - K^\dagger| + \alpha|K_t^{\text{raw}} - K^\dagger|$. Iterating yields the standard bound

$$|K_t - K^\dagger| \leq (1 - \alpha)^t |K_0 - K^\dagger| + \alpha \sum_{s=0}^{t-1} (1 - \alpha)^{t-1-s} |K_s^{\text{raw}} - K^\dagger|. \quad (11)$$

The first term decays geometrically; the second is controlled by the residual estimation error of K_s^{raw} .

Implications for the controller. This standard property has two consequences for Elastic Horizon. First, the steady-state horizon depends on the limiting target distribution of K_t^{raw} (which in turn depends on hyperparameters α, Δ), not on the initial horizon K_0 . This is consistent with the convergence experiments in Section 5.3, where runs starting from $K_0 = 10$ and $K_0 = 50$ both stabilize within the saturation band identified in Section 5.2. Second, the smoothing constant α trades responsiveness against variance: small α damps sampling noise in $\hat{B}_t$ but tracks capability changes more slowly, as confirmed empirically in Appendix F.

Limitation: non-stationary targets from policy updates. The above analysis assumes the input K_t^{raw} converges to a fixed target. In Elastic Horizon training, both the policy π_θ and the trajectory-length distribution F_L evolve over time, and the frontier H^* itself is dynamic (Section 4.1). The buffer $\mathcal{B}_t$ uses FIFO eviction precisely so that $\hat{B}_t$ tracks current rather than historical capability; as a consequence, K_t^{raw} is not a stationary input. The EMA bound above should therefore be read as a description of the controller’s *tracking* behavior under slow drift, not a claim of asymptotic convergence to a fixed point. Formal analysis of the joint policy–controller dynamics, including the cold-start and FIFO buffer effects, is left to future work.

C Algorithm Details

Algorithm 2 presents the complete Elastic Horizon training procedure with all implementation details.

Implementation notes.

- Training is implemented on top of the AgentEvolver agent training framework (Zhai et al., 2025). Elastic Horizon is added as a horizon-control module around the rollout loop (Phases 1, 3, and 4 above); the policy update path is left unmodified.
- The GRPO update follows Shao et al. (2024) with KL coefficient set to zero.
- The buffer $\mathcal{B}$ uses first-in-first-out (FIFO) eviction to maintain a sliding window of the most recent successful trajectory lengths, ensuring the estimate reflects current agent capability rather than historical performance.
- The floor operation $\lfloor \cdot \rfloor$ ensures integer horizon values.

Table 5: Complete hyperparameter settings.

Category	Parameter	Value
Training	Optimizer	AdamW
	Learning rate	1×10^{-6}
	Batch size	16
	Micro-batch size	8
	Grad accum steps	2
	Total training steps	200
GRPO	KL coefficient	0.0
	Discount factor γ	1.0
	GAE λ	1.0
	Clip ratio	0.2
Elastic Horizon	Initial horizon K_0	15
	Min. horizon $K_{\min}$	5
	Max. horizon $K_{\max}$	50
	EMA coefficient α	0.1
	Headroom Δ	10
	Buffer size N	100
	Min. buffer $N_{\min}$	20
Baselines	Linear inc. δ	0.2 per step
	TTI growth stages	15 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ 50
	TTI stage duration	50 steps
Evaluation	Rollouts per task	8
	Primary metric	mean@8

- During cold start ($|\mathcal{B}| < N_{\min}$), the horizon remains unchanged to avoid unreliable estimates from insufficient samples.
- We use $N_{\min} = 20$ to ensure the P90 estimate is based on at least 20 successful trajectories.

D Extended Experimental Details

D.1 Hyperparameter Settings

Table 5 provides the complete hyperparameter configuration for all experiments.

D.2 Environment Configuration

AppWorld (Trivedi et al., 2024): A high-fidelity execution environment simulating 9 daily applications (Amazon, Venmo, Spotify, Gmail, etc.) with 457 APIs. Tasks require agents to coordinate across multiple apps through API calls. We use the standard test split with binary sparse rewards (1 for task completion, 0 otherwise).

BFCL v3 (Patil et al., 2025): The Berkeley Function Calling Leaderboard evaluates multi-turn function calling across 8 API domains. The multi-turn split contains 1,000 test cases requiring stateful interactions with multiple function calls per task.

Both benchmarks enforce a maximum trajectory length of 50 steps. Trajectories exceeding the horizon constraint K_t are terminated with reward 0.

Table 6: Mean@8 with standard deviations across 3 random seeds (Qwen2.5-7B). Best in **bold**. *: training instability (highest variance, ± 4.52).

Method	AppWorld	BFCL
GRPO ($K=15$)	21.49 ± 2.14	66.87 ± 1.83
GRPO ($K=50$)	31.14 ± 3.07	$60.37 \pm 4.52^*$
TTI	37.28 ± 2.91	55.75 ± 3.68
ScalingInter	36.18 ± 2.53	69.12 ± 2.15
Elastic Horizon	39.47 ± 2.38	71.62 ± 1.96

D.3 Baseline Implementations

Linear schedule (ScalingInter-RL style):

$$K_t = \min(K_{\min} + \delta \cdot t, K_{\max}), \quad \text{with } \delta = 0.2. \quad (12)$$

This reaches $K_{\max} = 50$ at step $t = 200$.

Multiplicative schedule (TTI style): The horizon follows $K \in \{15, 20, 30, 50\}$ at training steps $\{0, 50, 100, 150\}$. This follows the stage-wise multiplicative curriculum of Shen et al. (2025), who raise the horizon at successive stage boundaries rather than by a fixed per-step increment; we start from the same initial horizon as the conservative fixed-horizon baseline ($K=15$) and cap the schedule at $K_{\max} = 50$.

Fixed horizon baselines: GRPO with $K = 15$ (conservative) and $K = 50$ (maximum), representing the extremes of fixed-horizon training.

D.4 Computational Resources

All experiments were conducted on a single compute node with 8 AI accelerator devices per training run. A 7B-parameter Elastic Horizon training run (200 training steps) requires approximately 320 device-hours of wall-clock time; a 14B run requires approximately 680 device-hours under the same settings. The Elastic Horizon controller itself adds negligible overhead ($< 0.1\%$) compared to trajectory sampling and gradient computation.

D.5 Statistical Significance

All main experiments are repeated with 3 random seeds. Table 6 reports mean@8 $\pm$ standard deviation across the 3 seeds for the 7B configuration, complementing the point estimates in Table 3. We do not report multi-seed variance for 14B due to compute constraints: each 14B training run consumes approximately 680 device-hours on 8 accelerator devices (~ 85 hours of wall-clock time per run), making 3-seed replication prohibitive.

Table 7: Quantile statistics of successful trajectory lengths at training step 100.

Benchmark	Min	Mean	P50	P90	Max
AppWorld	9	18.14	14	37.21	50
BFCL	1	11.93	11	24.29	50

Table 8: Percentile choice across benchmarks (Mean@8, %). P90 is the best statistic on both benchmarks.

Statistic	AppWorld SR (%)	BFCL SR (%)
P50	24.12	68.37
P90	39.47	71.62
P95	38.38	69.87
Max	36.0	62.88

E Trajectory Length Distribution Analysis

Observations supporting P90.

1. **Right skew:** Both distributions have positive skewness, confirming that trajectory lengths are heavy-tailed rather than symmetric.
2. **Mean underestimates capability:** The mean is substantially lower than P90, indicating that using the mean would set an overly conservative horizon that fails to accommodate harder task instances.
3. **Max is unstable:** The gap between P90 and Max reflects outlier trajectories from inefficient explorations. Using Max would inflate the horizon estimate and destabilize training.
4. **P90 balances coverage and robustness:** P90 covers 90% of successful trajectories while excluding the top 10% of potential outliers, providing a stable yet ambitious capability estimate.

F Additional Ablation Results

F.1 Full Percentile Comparison

To answer “why P90 over other percentiles?”, we report two complementary views: (i) the success rate achieved on each benchmark when the controller uses different statistics of $\mathcal{B}$ (Table 8); and (ii) the converged horizon, its stability, and the resulting interpretation on AppWorld (Table 9).

Performance increases monotonically from P50 to P90 on both benchmarks, then gradually decreases toward Max, consistent with the heavy-tail interpretation in Appendix B.

Table 9: Detailed percentile ablation on AppWorld (Step 200). We report success rate (SR), final converged horizon, and horizon stability (standard deviation over the last 50 steps).

Statistic	SR (%)	Final K	K Std
Mean	24.12	19.09	16.4
P50	24.12	15.82	14.9
P90	39.47	28.1	12.8
P95	38.38	44.15	9.7
Max	36.0	50.00	3.1

Table 10: Ablation on headroom Δ (AppWorld, Step 200).

Δ	SR (%)	Final K	Tokens
0	15.35	6.8	2.2×10^5
5	12.06	22.9	3.7×10^5
10	39.47	28.1	3.9×10^5
15	25.87	28.7	3.5×10^5
20	33.11	35.9	3.7×10^5

Analysis. Lower percentiles (Mean, P50) cause premature horizon convergence ($K \approx 15-19$), limiting the agent’s ability to solve harder tasks that require longer interactions. Higher percentiles (P95, Max) are sensitive to outlier trajectories, pushing K toward $K_{\max}$ and losing the efficiency benefit of adaptive control. P90 provides the best tradeoff: sufficiently high to accommodate challenging tasks, yet robust to occasional outliers.

F.2 Headroom Sensitivity

Table 10 provides complete results for the headroom ablation.

Analysis. Setting $\Delta = 0$ removes the exploration margin entirely: the target horizon can never exceed the current capability estimate, so the horizon collapses toward $K_{\min}$ ($K = 6.8$) and the agent never attempts tasks longer than what it already solves. Beyond this degenerate case, success rate is not monotone in Δ , and the gaps between neighbouring settings are comparable to the seed-to-seed spread reported in Table 6; we therefore claim only that $\Delta = 10$ is the best of the values we tried, not that a smooth trend exists. Larger headroom does raise the converged horizon ($K = 35.9$ at $\Delta = 20$), moving the controller toward fixed long-horizon behavior and eroding the efficiency benefit of adaptive control.

Table 11: Ablation on EMA coefficient α (AppWorld).

α	SR (%)	Final K	K Std	Behavior
0.1	39.47	28.1	12.8	Slow, stable
0.2	30.92	27.6	14.1	Mild oscillation
1.0	24.78	28	17.9	Unstable

F.3 EMA Coefficient Sensitivity

Analysis. The EMA coefficient α controls the tradeoff between responsiveness and stability. Small α (e.g., 0.1) yields very smooth trajectories but slow adaptation to capability changes. Large α (e.g., 1.0, equivalent to no smoothing) tracks the raw P90 estimate directly, causing oscillations due to sampling variance.

F.4 Sample Efficiency Comparison

Table 12 reports cumulative trajectory tokens consumed by each method on AppWorld at two operating points: (i) the first training step at which success rate reaches 50%, and (ii) the step at which each method attains its peak success rate. The “Token Savings” column is computed against GRPO ($K=50$) at the matched SR=50% operating point.

Analysis. Elastic Horizon reaches SR=50% with only 36.28M cumulative tokens, compared to 50.21M for GRPO ($K=50$), a **28% reduction at equivalent performance**. At peak performance, Elastic Horizon achieves the highest SR (56.87%) while consuming fewer total tokens than the strongest fixed-horizon baseline. Figure 5 visualizes the underlying trajectories: Elastic Horizon (blue) reaches matched SR with strictly fewer cumulative tokens than every fixed-horizon baseline. The 28% saving stems from two sources: shorter trajectories during early training (when the controller has not yet expanded to its converged value), and automatic convergence that avoids wasteful exploration beyond H^* . These two contributions correspond to the per-step gap visualized in Figure 4 of the main text.

BFCL average tokens per step. Figure 6 reports per-step token consumption on BFCL 7B for completeness. The trend mirrors AppWorld 14B: Elastic Horizon maintains lower per-step trajectory tokens than open-loop schedules throughout training. We move this panel to the appendix because the main-text efficiency story (Figure 4) is built around AppWorld 14B, where per-step gains are most pronounced.

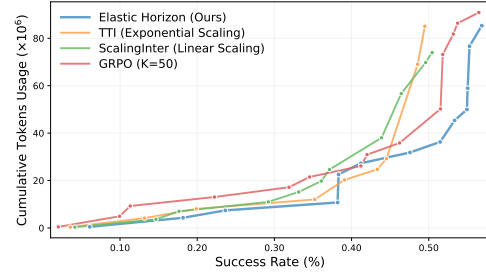


Figure 5: **Success rate vs. cumulative trajectory tokens (AppWorld, Qwen2.5-7B).** Elastic Horizon reaches matched success rates with strictly fewer cumulative tokens than every fixed-horizon baseline. At SR=50%, the cumulative gap is 28% vs. GRPO ($K=50$).

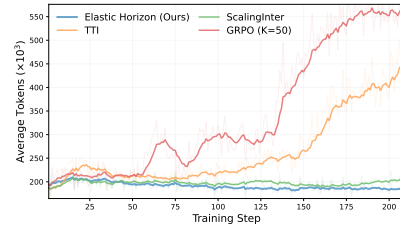


Figure 6: **Average tokens per training step on BFCL 7B.** Elastic Horizon (blue) consumes fewer per-step tokens than open-loop baselines throughout training, with the gap most visible during the rapid expansion phase of TTI/ScalingInter.

G Limitations and Future Directions

G.1 Cold Start Phase

During early training when successful trajectories are scarce, the P90 estimate may be unreliable. Our current solution (maintaining the initial horizon until buffer fills) is conservative but may delay adaptation. Future work could explore:

- Using trajectory lengths from *all* trajectories (not just successful ones) with appropriate weighting.
- Initializing the buffer with trajectories from a pretrained policy.
- Hybrid strategies that use open-loop schedules during cold start, then switch to closed-loop.

G.2 Single Global Horizon

Elastic Horizon adjusts a single global horizon K_t applied to all tasks. For task distributions with high variance in complexity, a single horizon may be suboptimal: easy tasks waste budget, while hard tasks may be truncated. Future work could explore:

- Per-task or per-difficulty-level horizon allocation.

Table 12: Sample efficiency on AppWorld (Qwen2.5-7B). Tokens are cumulative trajectory tokens (M = millions). Peak SR is the maximum Mean@8 attained during training. “—” indicates the method did not reach the SR threshold. Token savings are computed against GRPO ($K=50$) at the matched SR=50% operating point.

Method	Tok @ SR=50 %	Tok @ Peak	Peak SR (%)	Token Savings
GRPO ($K=50$)	50.21M	90.85M	56.49	(baseline)
ScalingInter (Linear)	73.98M	73.98M	50.44	—
TTI (Multiplicative)	—	85.07M	49.50	—
Elastic Horizon	36.28M	85.31M	56.87	~28%

- State-conditioned horizon prediction.
- Integration with task curriculum methods that also adapt task selection.

G.3 Non-Stationary Task Distributions

Our convergence analysis assumes the task distribution is stationary. If the distribution shifts during training (e.g., curriculum over tasks), the P90 estimate may lag. The FIFO buffer provides some adaptivity, but stronger distribution shift may require:

- Drift detection mechanisms.
- Weighted buffers that discount older samples.
- Explicit handling of task difficulty metadata.

G.4 Theoretical Gaps

The formal discussion in Appendix B is intentionally lightweight: we do not establish a closed-form bound between $\hat{B}_t$ and H^* , and we analyze the EMA update only under the simplifying assumption of a stationary target. Several theoretical questions remain open:

- Under what conditions is P90 *optimal* among percentile-based estimators?
- Can we derive the headroom Δ from first principles rather than tuning?
- How does the effective frontier H^* evolve during training under the joint policy–controller dynamics, and can this be characterized formally?

These questions present opportunities for future theoretical investigation.

H Extended Discussion

This appendix elaborates on two aspects of Elastic Horizon that are summarized only briefly in the main Conclusion.

What the frontier captures. The frontier H^* identified by Elastic Horizon reflects *computational complexity*—the number of interaction steps an agent needs to solve tasks—rather than *conceptual difficulty*. A task may be conceptually simple yet require many API calls (e.g., iterating over a list), or conceptually hard yet solvable in few steps (e.g., a single complex reasoning query). Elastic Horizon optimizes the former dimension because it directly determines training compute, and it does so without requiring practitioners to specify growth rates, stage boundaries, or maximum horizons, all of which depend on unknown task properties. This self-regulation is particularly valuable when deploying RL to new environments with unknown task complexity, when the task distribution is heterogeneous, and whenever compute is the binding constraint.

Complementarity with test-time horizon control.

Elastic Horizon operates during training to optimize sample efficiency, while concurrent test-time methods such as BATS (Liu et al., 2025) operate during deployment to optimize task performance under budget constraints. The two approaches address different stages of the agent lifecycle and are naturally complementary: combining training-time horizon adaptation with test-time budget awareness is a direction for end-to-end efficient agent systems.

Algorithm 2 Elastic Horizon Training (Complete)

Require: Initial policy π_{θ_0} , task distribution $\mathcal{T}$
Require: Initial horizon K_0 , horizon bounds $K_{\min}, K_{\max}$
Require: EMA coefficient α , headroom Δ
Require: Buffer capacity N , minimum buffer size $N_{\min}$
Require: Number of training steps T , batch size M

- 1: Initialize success buffer $\mathcal{B} \leftarrow \emptyset$
- 2: Initialize horizon $K \leftarrow K_0$
- 3: **for** training step $t = 1, 2, \dots, T$ **do**
- 4: *// Phase 1: Trajectory Collection*
- 5: Sample task batch $\{u_i\}_{i=1}^M$ from $\mathcal{T}$
- 6: **for** $i = 1, \dots, M$ **do**
- 7: Initialize trajectory $\tau_i \leftarrow (u_i)$
- 8: **for** interaction step $k = 1, \dots, K$ **do**
- 9: Sample action $a_k \sim \pi_{\theta_{t-1}}(\cdot | \tau_i)$
- 10: Execute action, receive observation o_k and done flag d_k
- 11: Append to trajectory: $\tau_i \leftarrow \tau_i \cup (a_k, o_k)$
- 12: **if** $d_k = \text{TRUE}$ **then**
- 13: **break**
- 14: **end if**
- 15: **end for**
- 16: Compute reward $r_i \leftarrow r(\tau_i)$ and length $L_i \leftarrow L(\tau_i)$
- 17: **end for**
- 18: *// Phase 2: Policy Update*
- 19: $\theta_t \leftarrow \text{GRPO}(\theta_{t-1}, \{(\tau_i, r_i)\}_{i=1}^M)$
- 20: *// Phase 3: Buffer Update (FIFO)*
- 21: **for** i such that $r_i = 1$ **do**
- 22: $\mathcal{B}.\text{APPEND}(L_i)$
- 23: **if** $|\mathcal{B}| > N$ **then**
- 24: $\mathcal{B}.\text{POPFIRST}()$
- 25: **end if**
- 26: **end for**
- 27: *// Phase 4: Horizon Update (Closed-Loop Control)*
- 28: **if** $|\mathcal{B}| \geq N_{\min}$ **then**
- 29: $\hat{B} \leftarrow \text{PERCENTILE}(\mathcal{B}, 90)$ $\triangleright$
- 30: Capability boundary estimate $K^{\text{raw}} \leftarrow \text{CLIP}(\hat{B} + \Delta, K_{\min}, K_{\max})$
- 31: $K \leftarrow \lfloor (1 - \alpha) \cdot K + \alpha \cdot K^{\text{raw}} \rfloor$ $\triangleright$
- 32: EMA smoothing **end if**
- 33: Log $(t, K, \hat{B}, \text{SR}_t)$
- 34: **end for**
- 35: **return** Trained policy π_{θ_T}
